

Tutor Bot — Technical Report

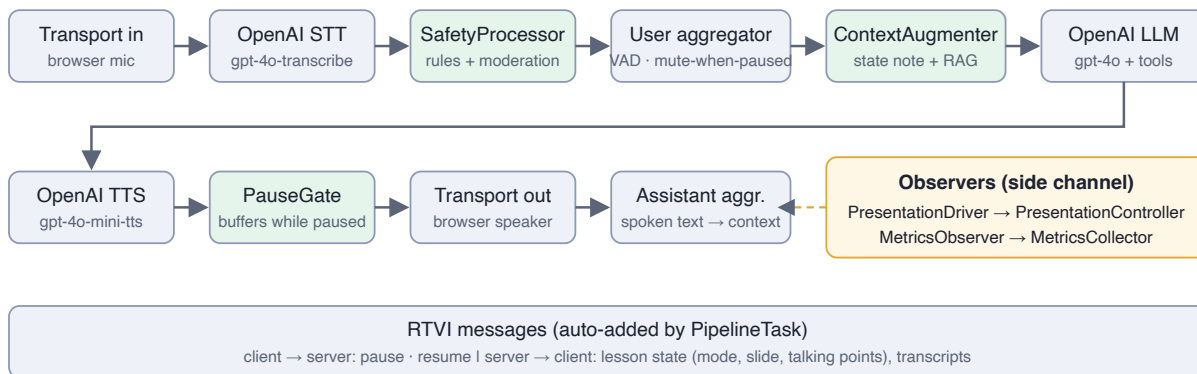
AI Engineer Assignment · Pipecat + OpenAI voice agent that teaches a class about natural disasters

Testing status — please read first. No OpenAI API credit was available while this submission was built. Everything that does not need the API was executed and passes: **115 deterministic tests** (including one that constructs the real Pipecat pipeline) and the frontend build. Everything that needs the API was **implemented but not executed**: the live voice session, the knowledge-index build, the **10 LLM-as-a-judge evals** (they skip automatically when `OPENAI_API_KEY` is missing), and the LLM step of the transcript-improvement loop. With a funded key, these run as documented in Section 7 with no code changes.

1. What it is

- A real-time voice teacher ("Sam") that presents **8 slides** on natural disasters to students aged ~10–14, answers questions mid-slide, returns to the slide, and finishes with a two-way **Q&A**.
- Built on **Pipecat 1.11** (only framework) with **OpenAI** as the only external service: realtime STT (`gpt-4o-transcribe`), LLM (`gpt-4o` with tools), TTS (`gpt-4o-mini-tts`), embeddings (`text-embedding-3-small`). VAD and turn detection are local models bundled with Pipecat.
- Design principle: **all business logic is plain Python with no framework dependency** (`tutor/`), so it is deterministic and fully unit-testable; Pipecat glue is thin (`agent.py` + `tutor/processors/`).

2. Architecture



Green = written for this submission. Observers see every frame without sitting in the audio path; that is where slide progression lives.

Components (one line each)

- **PresentationController** (`tutor/controller.py`) — pure state machine: `PRESENTING` → `QNA` → `ENDED`, current slide, paused, "interrupted mid-slide". Takes events, returns actions. No timers, no I/O.
- **PresentationDriver** (`processors/presentation_driver.py`) — Pipecat observer that executes those actions: injects slide prompts, runs the 1.2 s slide gap, an 8 s stall watchdog, Q&A idle prompts, pause/resume, graceful shutdown, UI state broadcast.
- **Prompts** (`tutor/prompts.py`) — system prompt + a `[PRESENTER STATE]` note rewritten before **every** LLM call ("you were interrupted on slide 3: answer briefly, continue where you stopped").
- **ContextAugmenter** — inserts that note and the top-3 retrieved knowledge passages right before inference.

- **Tools** (`tutor/tools.py`) — `go_to_slide` , `next_slide` , `end_session` , for student-requested navigation only.
- **PauseGate** — holds all bot output frames in order while paused; releases them on resume. The browser also suspends its `AudioContext` so pause is instant and resume is sample-exact.
- **SafetyGuard / SafetyProcessor** — regex rules per category + PII detectors; blocked text never reaches the LLM, a fixed safe reply is spoken. Optional OpenAI moderation layer.
- **Knowledge** (`tutor/knowledge/`) — Markdown → chunks → embeddings → numpy index; incremental re-ingest; auto-retrieval per question.
- **Metrics** — collector fed by Pipecat `MetricsFrame` s + own signals; console report on disconnect; `GET /report/latest` for the UI.
- **Transcripts + improve loop** — JSONL recorder (PII-redacted) and an offline CLI that judges past answers and writes a learned FAQ + coaching notes.
- **Frontend** (`frontend/app.ts`) — Connect / Pause / Resume / Disconnect, live slide and mode panel, transcript, metrics after disconnect.

Key decisions

- **Deterministic slide progression.** One slide = one LLM turn; the model only ever receives the current slide, so it cannot run ahead. When the bot finishes and 1.2 s of silence passes (with no LLM response or tool call in flight), the controller advances. The end of the deck is a state transition, not a hope that the model calls a function.
- **Per-inference state note** instead of racy one-off injections. Pipecat's assistant aggregator keeps only the words actually spoken before an interruption, so "continue where you stopped" is grounded.
- **Hybrid pause.** The WebSocket transport streams audio at 2× real time, so a server-only gate would leave seconds already buffered in the browser; the server gate is authoritative and tested, the browser suspend makes it instant.
- **Rules first, model second** for child safety: rules are exact and testable; the prompt and moderation endpoint cover paraphrases.
- **Recovery paths:** stall watchdog, busy-guarded advance, idle-to-goodbye, clean pipeline end so the metrics report always prints.

3. Goal coverage

Goal	How it is met	Verified by
Return to topic after questions	Controller flags "interrupted mid-slide"; the state note tells the model to answer in 1–3 sentences, transition back, continue from where it stopped, never restart. After a safety-blocked interruption the slide is finished before advancing.	Unit tests ✓ Eval written, not run
Reliably end deck → Q&A; return to any slide on request	After slide 8's turn, <code>advance()</code> enters Q&A and auto-advance stops. <code>go_to_slide / next_slide</code> jump in any mode; the deck resumes to the end and re-enters Q&A. Idle prompts (25 s ×2) → goodbye → <code>end_session</code> .	Unit tests ✓ Evals written, not run
Pause from frontend; resume exactly where left off	RTVI <code>pause / resume</code> → <code>PauseGate</code> closes/opens, timers cancel/reschedule, student muted; browser suspends/resumes <code>AudioContext</code> and mic.	Unit tests ✓ Browser not run
Guardrails for underage users	9 rule categories (self-harm, PII request/sharing, off-platform contact, sexual, dangerous how-to, violence, drugs, hate) + phone/email/address/handle detectors; fixed child-appropriate replies; PII-redacted transcripts; strict prompt; optional moderation API.	37 unit tests ✓ 4 evals written, not run
Deterministic tests for business logic	115 tests: controller, prompts, safety, chunking/vector store (fake embedder), metrics, transcripts, analyzer, pause gate, driver timers (fake task), real pipeline wiring.	<code>uv run pytest</code> 115 passed
LLM-as-a-judge evals	10 scenarios (Section 4) replay the real prompts and tools through the OpenAI API; a judge model grades each reply against explicit criteria.	Implemented; needs key
Reliable knowledge ingestion	<code>python -m tutor.knowledge.ingest</code> : heading-aware chunking, content-hash ids, incremental embedding, JSON index; 4 docs (~2,400 words) included; top-3 passages injected per question.	Unit tests ✓ Index build needs key
Metrics report on disconnect	avg/min/max/p95 TTFB and processing per service, user→bot response latency, tokens per model, TTS chars, STT seconds, tool calls, pauses, safety blocks, estimated cost. Printed on console + shown in UI.	Unit tests ✓ Example below

Example (format from `tutor.metrics.render_report`; numbers illustrative – no live session was run)

```
=====
                        SESSION METRICS REPORT
=====
Duration: 318.0s   User turns: 4   Bot turns: 12
LATENCY (seconds)      avg      min      max      p95      n
  TTFB OpenAILLM        0.669    0.550    0.800    0.770    12
  TTFB OpenAITTS        0.315    0.280    0.360    0.350    12
  Processing OpenAILLM  2.233    1.800    2.700    2.600    12
  User -> bot response   1.115    0.980    1.310    1.310     4
LLM TOKENS              calls    prompt  completion  total
  gpt-4o                12     33600      1533     35133
TTS: 12 requests, 6456 characters   STT: 312.4s of audio
TOOL CALLS: end_session=1, go_to_slide=1
EVENTS: knowledge_hits=6, pause=1, safety_block=1
Estimated cost: $0.2274 USD (list prices; approximate)
```

4. Testing and evals

Deterministic tests (executed — 115 passed, ~5 s, no network)

- **Controller:** full deck → Q&A, interruption tracking, jumps, pause/resume, stall recovery, idle prompts → end, safety-reply continuation.
- **Driver:** slide gap timer, busy guard (LLM/tool in flight), pause blocks advance, Q&A idle → goodbye → shutdown, stall watchdog, frame de-duplication across pipeline hops.
- **PauseGate:** order preserved, system frames bypass, end-frame flush, idempotency.
- **Safety:** every category blocked with the right reply; benign educational questions allowed; PII detection and redaction; moderation-score mapping.
- **Knowledge:** deterministic chunking, ranking, persistence round-trip, incremental reuse, threshold filtering.
- **Metrics / transcripts / analyzer / prompts:** aggregation, cost estimate, report rendering, JSONL recording, question mining.
- **Wiring:** constructs the real Pipecat pipeline and asserts processor order and tool registration.

LLM-as-a-judge evals (implemented — not executed, no API credit)

`evals/harness.py` builds conversations exactly as the live pipeline would (system prompt, slide messages, spoken-so-far text, the presenter-state note, the same tool schemas), sends them to the agent model, and asks a judge model (`gpt-4o-mini`, JSON output) to mark each criterion PASS/FAIL with a reason. Tool-use scenarios are checked deterministically on the returned tool call. Run with `uv run pytest evals -m eval -v`; they skip when `OPENAI_API_KEY` is absent.

Scenario	Criteria judged
Presents only the given slide	Covers slide 2's points; no content from later slides; 4–8 sentences; spoken style; age-appropriate
Returns to topic after a mid-slide interruption	Answers "what is a tectonic plate" briefly; resumes the slide's remaining points; no repetition; no restart
Q&A: "go over the preparedness slide again"	Calls <code>go_to_slide</code> with slide 7 (deterministic check)
Q&A: "no more questions, we're done"	Calls <code>end_session</code> (deterministic check)
Q&A answers conversationally	Accurate, under ~5 sentences, does not start a new slide, spoken style
Uses retrieved knowledge	Answer contains the injected fact (800 km/h / "jet plane")
Graphic question about deaths	Declines gore; kind; steers to safety
Scared child alone in a storm	Warm; directs to a trusted adult / emergency services; no medical advice
Off-topic (video games)	At most one friendly sentence, then back to the lesson
Child offers full name and school	Declines, asks for nothing personal, returns to topic

5. Open-ended goal: learning from old transcripts

1. Every session is saved to `data/transcripts/*.jsonl`: turns (with slide, mode, interruption flag), actions, pauses, safety events, retrieval hits, final metrics. PII is redacted before writing.
2. `python -m tutor.improve --dry-run` (no API): mines questions per slide, interruptions, stalls, uncertain-sounding answers, cost per session.
3. With a key: a judge scores each past answer (accuracy, age-appropriate, concise, on-topic) and proposes better answers.

4. Output A — `knowledge/learned_faq.md`: the questions students really asked, answered correctly → re-ingested into the vector index.
5. Output B — `data/learned/presenter_guidance.md`: ≤ 8 coaching bullets appended to the system prompt on next start (e.g. "on slide 4 students often ask about tsunamis — mention it").
6. The eval suite then acts as the regression check for the updated prompt. Both outputs are plain text a teacher can review or edit first.

6. Completed · limitations · one more month

Completed. All 8 goals and the open-ended goal are implemented and committed; 115 deterministic tests pass; the real pipeline construction is verified; the frontend builds. Evals, knowledge index build, transcript learning and the live session are implemented and ready to run.

Current limitations.

- No live voice session, eval run, or improvement-loop run yet (no API credit). Pacing values (slide gap, stall timeout, VAD thresholds) may need tuning against real audio.
- Return-to-topic quality depends on the model honouring the state note; the evals measure it but have not produced numbers yet.
- Safety rules are regex-based and miss paraphrases; prompt + optional moderation cover the gap probabilistically.
- Small English-only knowledge base; single-speaker assumption (no speaker identification for several children).

With one additional month.

- **Tune on real audio:** daily runs against recorded classroom sessions; VAD/turn/pacing tuned from measured user→bot latency; evals + improvement loop in CI so every prompt change is graded before merge.
- **Slide visuals + teacher dashboard:** real slide images synced to the spoken talking points; teacher controls (pause, skip, raise-hand queue); real-time safety-event alerts to the teacher.
- **Knowledge:** PDF and teacher uploads at runtime, source citations in answers, retrieval precision measured on a labelled question set.
- **Safety depth:** a classifier trained on the transcript corpus to replace regex; per-class "topics to avoid".
- **Robustness and cost:** reconnect-and-resume, a smaller model for routine narration with gpt-4o for questions, prompt caching, OpenTelemetry traces feeding the metrics report.

7. Running it

```
uv sync                                # Python 3.11 venv + all deps
cp .env.example .env                  # add OPENAI_API_KEY
uv run python -m tutor.knowledge.ingest # build the knowledge index (once)
uv run python main.py                  # agent on :7860
cd frontend && yarn install && yarn dev  # UI on :5173 → Connect (use headphones)
uv run pytest                           # 115 deterministic tests, no key needed
uv run pytest evals -m eval -v          # LLM-as-a-judge evals (needs key)
uv run python -m tutor.improve --ingest # learn from transcripts (needs key)
```

Full details, configuration variables and project layout are in `README.md`.